ProDebugger: A Programmable Platform for Evidence-Grounded Debugging of LLM Agent Trajectories

Jintian Hu*
JINTIAN002@e.ntu.edu.sg
Nanyang Technological University

Yuyang Li*
E1519922@u.nus.edu
National University of Singapore

Chuyi Deng
DENG0167@e.ntu.edu.sg
Nanyang Technological University

Tianyi Tang
Tang_Tianyi@a-star.edu.sg
CFAR, A*STAR, Singapore

Jie-Jing Shao
shjjhszs@gmail.com
CFAR, A*STAR, Singapore

An Li
AN007@e.ntu.edu.sg
Nanyang Technological University

Ivor Tsang
ivor_tsang@a-star.edu.sg
CFAR, A*STAR, Singapore

Haiyan Yin✉
yin_haiyan@a-star.edu.sg
CFAR, A*STAR, Singapore

Abstract

LLM agents solve complex tasks through long sequences of searches, tool calls, and environment interactions, yet a failure may surface many steps after the decision that introduced it. We present ProDebugger[1], a programmable platform for evidence-grounded debugging of LLM agent trajectories. It compiles free-form traces into a typed record of task requirements, entities, state changes, and source-linked facts. A read-only domain-specific language lets debugging agents query this record across nonadjacent steps while preserving the source span behind each result. An agentic workflow uses these queries to compare candidate explanations under explicit evidence checks and returns the best-supported step, implicated module, fault class, and cited evidence. The interface exposes executed queries and their original trace spans, allowing developers to inspect or redirect the investigation. We demonstrate this process in an ALFWorld scenario, where ProDebugger links an unmet requirement to an earlier identity error rather than its downstream symptoms. The codebase includes additional compiled trajectories from ALFWorld and WebShop.

CCS Concepts

• **Information systems → Information systems applications**.

Keywords

LLM agents, agent debugging, failure attribution, trace analysis

ACM Reference Format:
Jintian Hu, Yuyang Li, Chuyi Deng, Tianyi Tang, Jie-Jing Shao, An Li, Ivor Tsang, and Haiyan Yin. 2027. ProDebugger: A Programmable Platform for Evidence-Grounded Debugging of LLM Agent Trajectories. In *Proceedings of the 20th ACM International Conference on Web Search and Data Mining (WSDM '27), February 15–19, 2027, Hong Kong, China.*

1 Introduction

When an agent run fails, developers usually begin with its execution log. Observability tools organize activity into chronological traces or structured events [1, 5], but evidence explaining a failure often spans nonadjacent steps. For a web shopping agent, a user constraint may guide the initial search but be lost when comparing products, leading to an incompatible purchase. Diagnosis therefore requires connecting requirements, retrieved information, and decisions, distinguishing an initiating error from its downstream effects, and keeping missing support distinct from a violation [8–10]. This leads to a practical question:

> **Which decision best explains the failure, and which trace evidence supports that explanation?**

ProDebugger turns this investigation into a programmable platform for automated analysis and human inspection. Domain profiles compile tasks and traces into typed facts for requirements, entities, states, and actions, with links to their original sources. Rule checks identify investigation leads that debugging agents examine through a read-only declarative domain-specific language, connecting evidence across steps. Executed queries, returned facts, and source spans remain visible as shared workflow state. In the demonstration, attendees can switch between WebShop and ALFWorld cases within the same workbench, inspect how candidate diagnoses were formed, and revisit the underlying source steps through a navigable evidence chain (Figure 2).

2 Related Work

Observability makes an agent run visible; diagnosis must connect the observed outcome to preceding decisions. AgentOps [5] identifies lifecycle artifacts for monitoring, while AgentTrace [1] structures operational, cognitive, and contextual events as telemetry. Who&When [8] benchmarks attribution to responsible agents and decisive steps in multi-agent runs. Diagnostic systems approach the problem from complementary angles: AgentDebug [9] uses a failure taxonomy, AgentDebugX [10] couples multi-turn investigation with recovery and rerunning, and AgentRx [3] synthesizes constraints to localize critical steps. ProDebugger addresses the operational gap between recorded telemetry and diagnostic judgment. It turns a completed run into typed evidence, retrieves that evidence through explicit programs, and keeps each returned fact tied to its source span. A query can therefore be replayed independently of the workflow conclusion it informs.

*These authors contributed equally to this work and share first authorship.
[1]Source code: https://github.com/GaloisH/ProDebugger

Explicit specifications provide another route to structured analysis. RML [2] defines expected behavior over event traces for runtime verification, while AgentSpec [7] and Agent-C [6] encode constraints that detect or prevent non-compliant actions. HarnessFix [4] compiles trajectories and harness artifacts into an intermediate representation before attributing failures and generating repairs. ProDebugger places declarative structure at the evidence-access boundary: its read-only programs retrieve states, actions, and source spans after execution. The returned rows become workflow state while retaining the program that produced them. This design connects structured telemetry to diagnostic deliberation without conflating evidence retrieval with model judgment.

3 System Architecture

ProDebugger preserves a continuous, source-linked evidence path from a failed trajectory to a reviewable diagnosis (see Figure 1). The compiler builds a typed record, rule checks surface investigation leads, programmable queries assemble evidence across steps, and an agentic workflow compares the resulting explanations. The submitted diagnosis remains linked to the queries, facts, and source spans from which it was formed, allowing the investigation to be replayed against the original trajectory.

3.1 A Debugging View of Agent Traces

ProDebugger captures four facets of an agent trace:

- **Task intent.** The instruction, subgoals, and constraints that define what the run was expected to accomplish.
- **World state.** Entities, relations, and observations that record what was true, or remained unknown, at each step.
- **Agent state.** Memories, reflections, and plans that expose what the agent believed and how those beliefs changed.
- **Execution evidence.** Actions, outcomes, source spans, and provenance links that connect a decision to its observable effects.

Keeping these views separate locates a failure in task interpretation, agent belief, or execution. The same facets define the compiled schema in §3.2 and the query targets in §3.3, so each diagnostic claim remains tied to the kind of evidence it requires.

3.2 Parsing and Rule Checking

The compiler realizes this debugging view as a record that later stages can inspect. Parsing extracts typed facts from each step; rule checking reads those facts cumulatively to surface possible breaks in state, action, or task progress.

Parsing. We represent a trajectory as $\tau = (I, x_1, \ldots, x_T)$, where I is the task instruction and each step x_t contains an observation, memory, reflection, plan, and action. A domain profile defines the permitted entity types, identifiers, and relations, together with properties such as arity, mutability, and functionality. An LLM-assisted parser π_{parse} maps each step to structured facts:

$$F = \bigcup_{t=1}^{T} \pi_{\text{parse}}(x_t). \tag{1}$$

Each fact is represented as $f = \langle rel, args, value, slot, side, t, basis, span \rangle$. It records a relation and its arguments, the asserted value, source module, assertion origin, time step, provenance, and original text span. Observations and memories share this schema so that their claims can be compared; actions additionally produce admissibility facts. Every fact must reference known entities, preserve its source span, and link only to earlier supporting facts. Unresolved information remains unknown. Together, the parsed entities, relations, and facts form the compiled trajectory record D.

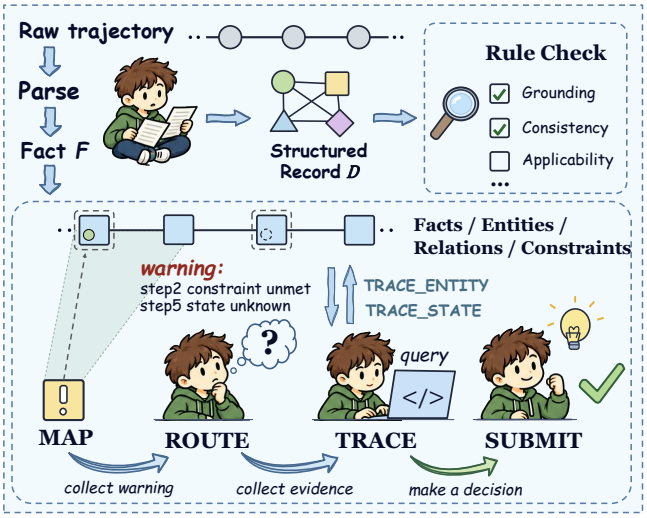


Figure 1: ProDebugger turns a raw trajectory into a reviewable diagnosis. Parsing creates a typed, source-linked record; rule checks surface investigation leads; programmable evidence retrieval gathers cross-step support; and an agentic workflow deliberates over competing explanations. The selected diagnosis retains its queries and source spans for replay and inspection.

Rule Checking. The checker reads D cumulatively: at step t, it can use only facts recorded through that step. It applies seven rules:

- **R1 Consistency** flags memory that conflicts with observation.
- **R2 Grounding** flags observation that contradicts a memory claim.
- **R3 Update** flags memory that retains an invalidated value.
- **R4 Applicability** flags an inadmissible attempted action.
- **R5 Execution** flags an executed action with no effect.
- **R6 Completion** flags a claimed but unsatisfied goal.
- **R7 Progress** flags an action repeated with the same outcome.

Each check returns *satisfied*, *violated*, *unknown*, or *not applicable*. Missing evidence therefore yields *unknown*, not a violation. A violation is an investigation lead rather than an attribution. These leads become query targets in §3.3, where the debugger tests them against evidence from other steps before the workflow compares candidate explanations.

3.3 Program-Guided Evidence Inspection

Programmable evidence retrieval moves investigation beyond linear log reading and one-shot judgment over a full trace: a debugging agent can isolate relevant facts, test relations across steps, and verify each result against its source span. ProDebugger realizes this mechanism through a read-only domain-specific language (DSL).

The manager assigns querying agent i a task q and a fact subset $F_i \subseteq F$. Conditioned on these inputs, the agent generates a DSL program and executes it over the compiled record:

$$\underbrace{p_i = \pi_{\text{tracer}}(F_i, q)}_{\text{Generate a query program}} \quad \Longrightarrow \quad \underbrace{V_i = \text{Execute}(D, p_i)}_{\text{Execute to retrieve evidence}} . \tag{2}$$

Here, π_{tracer} is the query-generation policy, p_i the program, and D the compiled record. The agent combines retrieved evidence V_i with F_i to produce a step-level candidate.

Built-in functions (Table 1) provide entry points for tracing entities, states, and actions, retrieving source passages, and following fact provenance. Agents parameterize these functions with investigation targets, such as entity names, relations, or fact identifiers. Agents compose the returned rows with `SELECT`, `JOIN`, and `WHERE` to express conditions across records.

Table 1: Built-in functions for tracing entities, states, actions, source passages, and fact links.

Function	Effect
TRACE_ENTITY	Retrieve records about entity e.
TRACE_STATE	Trace recorded assignments of relation r for entity e.
TRACE_ACTION	Retrieve recorded actions of type a.
SEARCH_TEXT	Find phrase s in source field m.
SOURCE	Retrieve a passage matching s from fact f's source.
TRACE_FACT	Follow recorded provenance links from fact f.

For example, consider the question: *Does the recorded state support the agent's claim that the cup is clean?* Listing 1 links memories about cleanliness to clean-state facts recorded at or before each claim; unmatched claims remain unknown.

Listing 1: A programmable cleanliness check

```
-- Retrieve cup memories with source references.
SELECT c.text, s.value, c.source_id, s.fid
FROM SEARCH_TEXT('cup 1','memory') c
-- Join clean-state facts up to each claim; keep unknowns.
LEFT JOIN TRACE_STATE('portable:cup_1','has') s
  ON s.step <= c.step
 AND json_extract(s.args,'$[1]')
     = 'attribute:clean'
-- Focus on memories mentioning the cup's cleanliness.
WHERE lower(c.text) LIKE '%clean%';
```

The DSL makes evidence selection part of the investigation record. Agents compose typed operators over trace entities, states, actions, sources, and fact links, then replay or revise the program for another hypothesis. Execution is read-only over D, and returned rows retain fact identifiers and source spans. Developers can inspect both the evidence and its selecting program before workflow deliberation.

3.4 Evidence-Grounded Agentic Workflow

The workflow compares candidate steps and modules using the evidence bundles produced by executable queries. A manager coordinates four deliberative stages. **(i) MAP** turns unmet goals into explicit investigation targets. **(ii) ROUTE** follows supporting facts and rule violations backward from each target to a small set of *anchor* steps. **(iii) TRACE** turns each anchor into a question and assigns it, together with a fact subset F_i, to a tracing agent; the agent executes one DSL program and returns one candidate step with its source spans. **(iv) SUBMIT** compares the candidates and returns the best-supported attribution (*step*, *module*, *fault class*, *evidence*). The record and DSL determine what can be cited; the LLM compares candidates within that evidence, and every cited fact and span must exist in the record.

The manager compares evidence bundles rather than free-form explanations. When support for a candidate is incomplete or conflicts with another bundle, it can issue a follow-up question over the same compiled record. Each round appends its program, returned rows, and source spans to the workflow state. This selective, multi-round access limits repeated full-trace prompting and keeps every refinement available for inspection.

4 Demonstration

Figure 2 links the unmet requirement to the submitted candidate through shared identifiers, making the investigation navigable in reverse.

4.1 An Evidence-Linked Investigation Interface

(1) Requirement map. MAP turns the task into named requirements and assigns each one an evidence status. Its constraints lead directly to the observations or decisions used for that status, making an unmet requirement an investigation target rather than a detached error message. **(2) Rule checks.** Each step summarizes the symbolic rules evaluated there and the facts cited by every violation. **(3) Linked timeline.** Phase cards connect `PARSE`, `MAP`, `ROUTE`, `TRACE`, and `SUBMIT` to the agent steps they inspect. **(4) Query record.** The DSL panel preserves the manager's question, executed program, and returned rows. **(5) Evidence chain.** The step-indexed entity graph links those rows to fact histories and source identifiers. **(6) Submitted diagnosis.** The result view keeps the selected candidate beside the alternatives considered by the workflow. Because these views share identifiers, selecting a diagnosis recovers the requirement, query, facts, and source step that produced it. The interface also supports replay: rerunning a stored program against the same compiled record reconstructs its returned rows and source links, while a revised query continues the investigation from the same evidence state.

4.2 Demonstration Scenario

The demonstration follows a failed ALFWorld episode from an unmet requirement to the evidence used for attribution.

> **Demonstration task.** *Put a spraybottle in countertop.* The agent searches the cabinets for 30 steps but never observes a spraybottle.

Locate the decision. MAP derives one requirement constrained by object type and destination; Figure 2(1) marks it as *unmet*. The rule checks in Figure 2(2) expose only surface symptoms: at steps 10–11, the quoted action `'go to cabinet 3'` is inadmissible and has no effect, while step 14 raises no violation. Guided by the unmet requirement, ROUTE anchors step 14, where the agent takes `soapbottle_1` from cabinet 3 (Figure 2(3)).

Test the identity premise. MANAGER asks where the mistaken identity entered the run, and TRACE executes one DSL program over

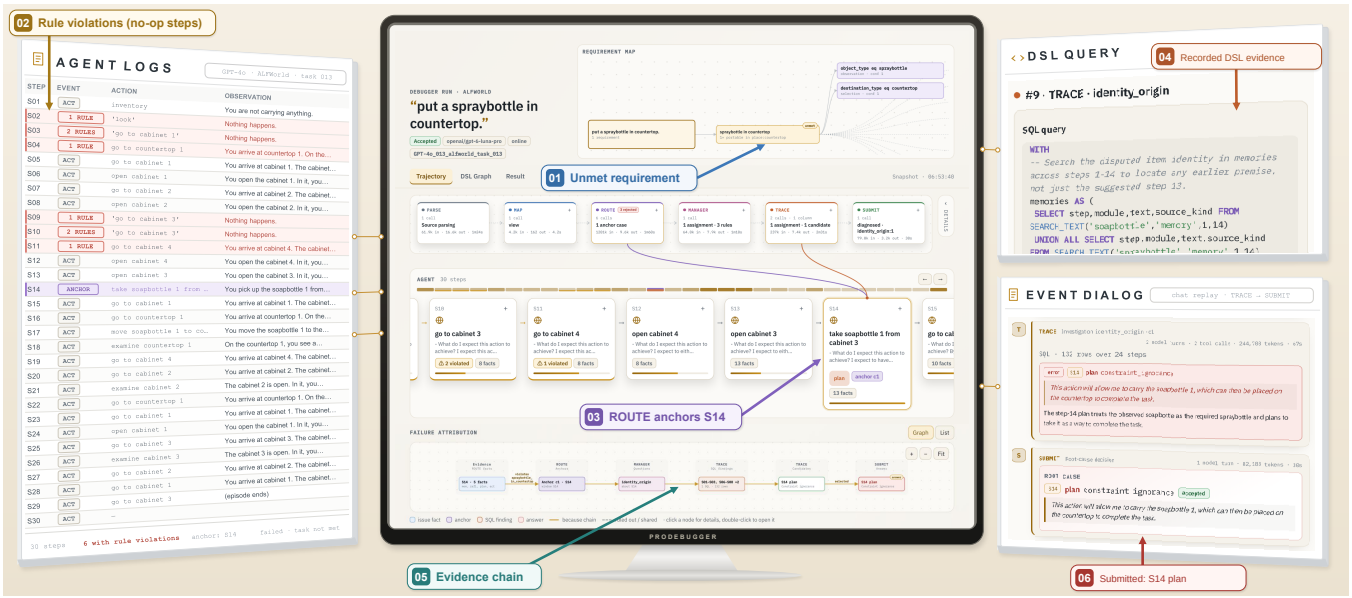


Figure 2: The ProDebugger interface closes the path from failure to reviewable diagnosis. (1) MAP exposes the unmet requirement; (2) rule checks surface no-op symptoms; (3) ROUTE anchors the relevant step; (4) TRACE records the executed DSL query and its evidence; (5) linked views preserve the evidence chain; and (6) SUBMIT returns the best-supported candidate. Shared identifiers let users move from the submitted diagnosis back to the source log.

the evidence from steps 1–14 (Figure 2(4)). The query surfaces the step-14 plan, which treats the observed soapbottle as the required spraybottle. The entity graph records `soapbottle_1` in cabinet 3 but contains no fact supporting that equivalence.

Review and redirect. Submit compares the resulting evidence bundle with the other candidates and returns the step-14 plan as the best-supported explanation (Figure 2(6)); the later placement remains a downstream consequence. ProDebugger therefore offers a replayable basis for failure attribution: every candidate remains linked to the queries, facts, and source spans that support it. The platform keeps this evidence path open to human direction, allowing developers to inspect, replay, and redirect the investigation interactively.

Additional case studies are available on our project page.

5 Conclusion

We introduced ProDebugger, a programmable platform for investigating failed agent trajectories. It compiles runs into a typed, source-linked record, uses rule checks to surface investigation leads, and provides a read-only DSL for assembling evidence across non-adjacent steps. An agentic workflow compares evidence bundles before submitting its best-supported diagnosis. The interface presents the same queries, facts, and source spans to developers, allowing them to inspect, replay, and redirect the investigation. These capabilities support post-training analysis and enterprise incident review, where teams must document why a run failed and assess a proposed repair before redeployment. Diagnoses remain bounded by extraction quality and evidence coverage, with unsupported claims reported as *unknown* or *uncertain*. Future work will evaluate developer interaction and intervention-based causal validation.

References

[1] Adam AlSayyad, Kelvin Yuxiang Huang, and Richik Pal. 2026. AgentTrace: A Structured Logging Framework for Agent System Observability. arXiv:2602.10133. https://arxiv.org/abs/2602.10133

[2] Davide Ancona, Luca Franceschini, Angelo Ferrando, and Viviana Mascardi. 2021. RML: Theory and Practice of a Domain Specific Language for Runtime Verification. *Science of Computer Programming* 205 (2021), 102610. doi:10.1016/j.scico.2021.102610

[3] Shraddha Barke, Arnav Goyal, Alind Khare, Avaljot Singh, Suman Nath, and Chetan Bansal. 2026. AgentRx: Diagnosing AI Agent Failures from Execution Trajectories. arXiv:2602.02475. https://arxiv.org/abs/2602.02475

[4] Mengzhuo Chen, Junjie Wang, Zhe Liu, Yawen Wang, Haiming Zheng, and Qing Wang. 2026. From Failed Trajectories to Reliable LLM Agents: Diagnosing and Repairing Harness Flaws. arXiv:2606.06324. https://arxiv.org/abs/2606.06324

[5] Liming Dong, Qinghua Lu, and Liming Zhu. 2024. AgentOps: Enabling Observability of LLM Agents. arXiv:2411.05285. https://arxiv.org/abs/2411.05285

[6] Adharsh Kamath, Sishen Zhang, Calvin Xu, Shubham Ugare, Gagandeep Singh, and Sasa Misailovic. 2025. Enforcing Temporal Constraints for LLM Agents. arXiv:2512.23738. https://arxiv.org/abs/2512.23738

[7] Haoyu Wang, Christopher M. Poskitt, and Jun Sun. 2026. AgentSpec: Customizable Runtime Enforcement for Safe and Reliable LLM Agents. In *Proceedings of the 48th IEEE/ACM International Conference on Software Engineering*. 2938–2950. doi:10.1145/3744916.3764546

[8] Shaokun Zhang, Ming Yin, Jieyu Zhang, Jiale Liu, Zhiguang Han, Jingyang Zhang, Beibin Li, Chi Wang, Huazheng Wang, Yiran Chen, and Qingyun Wu. 2025. Which Agent Causes Task Failures and When? On Automated Failure Attribution of LLM Multi-Agent Systems. In *Proceedings of the 42nd International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 267)*. 76583–76599. https://proceedings.mlr.press/v267/zhang25cq.html

[9] Kunlun Zhu, Zijia Liu, Bingxuan Li, Muxin Tian, Yingxuan Yang, Jiaxun Zhang, Pengrui Han, Qipeng Xie, Fuyang Cui, Weijia Zhang, Xiaoteng Ma, Xiaodong Yu, Gowtham Ramesh, Jialian Wu, Zicheng Liu, Pan Lu, James Zou, and Jiaxuan You. 2025. Where LLM Agents Fail and How They Can Learn From Failures. arXiv:2509.25370. https://arxiv.org/abs/2509.25370

[10] Kunlun Zhu, Xuyan Ye, Zhiguang Han, Yuchen Zhao, Bingxuan Li, Weijia Zhang, Muxin Tian, Xiangru Tang, Pan Lu, James Zou, Jiaxuan You, and Heng Ji. 2026. AgentDebugX: An Open-Source Toolkit for Failure Observability, Attribution, and Recovery in LLM Agents. arXiv:2607.18754. https://arxiv.org/abs/2607.18754